CẨM NANG ÔN TẬP GIỮA KỲ

Lập trình Web — HTML · PHP · Vue 3 · MySQL

Biên soạn từ 7 file slide buổi 1, 1b, 2, 3, 4, 5, 6 và đề mẫu 14 câu. Trọng tâm: PHP (cú pháp, OOP, PDO, form, session), Vue (directives, reactive, lifecycle), HTML (DOM/form) và MySQL. Các ví dụ bổ sung để luyện thi được ghi rõ.

Phần	Độ ưu tiên	Phạm vi
PHP	RẤT CAO	Biến, toán tử, mảng, hàm, scope, OOP, include/require, exception, PDO, CRUD, login
Vue 3	CAO	Reactive; các directive; v-model; methods/computed/watch/mounted
HTML	CAO	DOM, các thẻ, bảng, form, method/action/name, SSR/CSR
MySQL	VỪA	PK/FK, kiểu dữ liệu, CRUD, JOIN, GROUP BY/HAVING
HTTP + bảo mật	CAO	Request/Response, cookie/session, Injection, XSS, session fixation

CẦN NHỚ / RỦI RO — Đề mẫu có 14 câu; nhiều câu “chọn nhiều” kèm giải thích hoặc điền thêm. Hãy học theo cơ chế và biết nói “vì sao”, không chỉ nhớ định nghĩa. Quiz riêng được chuyển sang chuẩn import trắc nghiệm/checkbox.

1. KIẾN TRÚC WEB, HTTP, SSR/CSR (Buổi 1 + HTML 1b)

Luồng chuẩn: Browser (client) → HTTP Request → Apache/web server → PHP xử lý → (khi cần) MySQL → PHP tạo HTML hoặc JSON → HTTP Response → Browser. PHP chạy trên SERVER; Vue chạy trong TRÌNH DUYỆT. File index.php mở qua http://localhost/... thay vì file:// để PHP được thực thi.

Khái niệm	Ghi nhớ
HTTP GET	Dùng đọc/tìm kiếm; tham số thường nằm ở URL/query string
HTTP POST	Dùng gửi dữ liệu form trong request body; không hiện trên URL nhưng KHÔNG tự mã hóa
Request	Method, URL/path, headers, body (tùy method)
Response	Status 200/302/404/500, headers (Content-Type, Set-Cookie), body HTML/JSON
SSR	Server ghép HTML rồi gửi cho browser (ví dụ PHP render)
CSR	Browser dùng JavaScript/Vue để tạo/cập nhật giao diện; có thể lấy JSON qua API
DevTools	Network: request URL/method/status/headers/body; Elements: cây DOM đã chạy JS

CẦN NHỚ / RỦI RO — 200=OK, 302=redirect, 404=not found, 500=lỗi server. Status code nằm ở RESPONSE, không phải request. GET/POST không quyết định trang có reload hay không: form submit thường reload, fetch/AJAX có thể không.

2. HTML: CẤU TRÚC, FORM, DOM (Buổi 1b)

2.1. Bộ khung, thẻ và thuộc tính

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="utf-8">
  <title>Mini Shop</title>
</head>
<body>
  <h1>Danh sách s□n ph□m</h1>
  <p>Xin chào <strong>bạn</strong>.</p>
</body>
</html>
```

Thẻ/thuộc tính	Ý nghĩa và bẫy
html, head, body	head chứa metadata; body chứa nội dung nhìn thấy. UTF-8 giúp hiển thị tiếng Việt
h1–h6, p, strong, em	Tiêu đề, đoạn văn, nhấn mạnh mạnh/nhẹ
br, hr, img, input	Thẻ rỗng: không cần thẻ đóng; img cần src, nên có alt
ul, ol, li	Danh sách không thứ tự/có thứ tự; mục danh sách
table, tr, th, td	Bảng; tr là hàng; th là tiêu đề; td là ô dữ liệu
div và span	div thường dạng khối, span nội tuyến
id và class	id duy nhất trên một trang; class có thể dùng lại
href và src	href liên kết; src chỉ tài nguyên như ảnh
name và id	name quyết định tên dữ liệu gửi lên server; id để nhận diện/liên kết label

```
<table border="1">
  <tr><th>ID</th><th>Tên</th></tr>
  <tr><td>1</td><td>Bút bi</td></tr>
</table>
<ul><li>Bút</li><li>V□</li></ul>
<ol><li>Chọn</li><li>G□i</li></ol>
```

2.2. Form — phải thuộc cú pháp

```
<form action="luu.php" method="post">
  <label for="ten">Tên:</label>
  <input id="ten" name="ten" type="text" required>
  <input name="gia" type="number" min="1">
  <input name="id" type="hidden" value="5">
  <select name="danh_muc"><option value="1">Sách</option></select>
  <textarea name="mo_ta"></textarea>
  <label><input type="checkbox" name="con_hang"> Còn hàng</label>
  <button type="submit">Lưu</button>
</form>
```

action = nơi nhận dữ liệu; method = GET/POST; name = khóa được gửi; id = định danh của phần tử; label for phải trùng id; button type=submit kích hoạt form; textarea nhập nhiều dòng, select/option chọn giá trị. Checkbox không tích thường không xuất hiện trong dữ liệu POST.

CẦN NHỚ / RỦI RO — Quên name sẽ không có khóa $_POST["ten"]; type=password chỉ che ký tự trên màn hình, không mã hóa. required / min chỉ hỗ trợ phía client; server vẫn phải validate. Hidden input không hề bí mật, có thể bị sửa.

2.3. DOM, sự kiện và rendering

DOM = Document Object Model: HTML được duyệt thành cây nút cha/con. Lồng thẻ phải đúng, thẻ mở/đóng tương ứng, không trùng id. Tab Elements phản ánh DOM hiện tại; View Source có thể khác sau khi JS/Vue cập nhật.

Các event thường gặp: click, submit, input, change, mouseover, load. Tương tác nội bộ frontend khác với gửi request lên server. Trong Vue, @click/@submit dùng để bắt event.

CẦN NHỚ / RỦI RO — Không đặt tr trực tiếp bên ngoài table; không lồng sai thứ tự ... . Kiểm tra bằng validator.w3.org. Mọi dữ liệu không tin cậy đưa ra HTML phải được escape theo ngữ cảnh.

3. PHP CƠ BẢN — CÚ PHÁP PHẢI NHỚ (Buổi 1)

3.1. Biến, kiểu, hằng số, output, chuỗi

```
<?php
$ten = "An";        // string
$tuoi = 20;         // int
$diem = 8.5;        // float
$coHang = true;     // bool
$ds = [1, 2, 3];    // array
$rong = null;       // null
const VAT = 0.1;
define("ITEM_PER_PAGE", 5);
echo "Chào $ten";  // Chào An
echo 'Chào $ten';  // Chào $ten
echo "Tên: " . $ten;
var_dump($diem);   // float(8.5)
print_r($ds);      // xem c□u trúc m□ng
?>
```

PHP biến bắt đầu bằng $, phân biệt hoa/thường; hằng không dùng $. Dấu chấm (.) nối chuỗi; .= nối vào chuỗi có sẵn. Trong nháy kép, biến đơn giản được nội suy; trong nháy đơn thì không. echo xuất nội dung; return trả giá trị khỏi hàm.

3.2. Toán tử, so sánh, điều kiện

```
$a = 5; $b = "5";
var_dump($a == $b);   // true: so sánh giá trị
var_dump($a === $b);  // false: khác ki□u
$x = 7 % 2;            // 1: ph□n dư
$x += 3;               // 4
$ketQua = ($a >= 5 && $x > 0) ? "Đạt" : "Trượt";
if ($a >= 8) { echo "Gi□i"; }
elseif ($a >= 5) { echo "Đạt"; }
else { echo "Chưa đạt"; }
switch ($a) {
  case 5: echo "Năm"; break;
  default: echo "Khác";
}
```

Viết	Ý nghĩa
= / == / ===	Gán / so sánh lỏng / so sánh cả giá trị + kiểu
!= / !==	Khác lỏng / khác chặt
&& / || / !	AND / OR / NOT
%	Chia lấy dư; ví dụ $n % 2 === 0 kiểm tra số chẵn
??	Null coalescing: $x ?? "" lấy vế phải khi biến chưa có hoặc null
?:	Toán tử ba ngôi: điều kiện ? A : B
+= -= .=	Cộng, trừ, nối chuỗi rồi gán

3.3. Vòng lặp, break/continue

```
for ($i = 1; $i <= 3; $i++) { echo $i; } // 123
$n = 3;
while ($n > 0) { echo $n; $n--; }         // 321
$ds = ["Bút", "V□"];
foreach ($ds as $sp) { echo $sp; }
foreach ($ds as $i => $sp) { echo "$i: $sp"; }
for ($i = 1; $i <= 5; $i++) {
  if ($i === 2) continue; // b□ lượt i=2
  if ($i === 4) break;    // dừng h□n
  echo $i;                // 13
}
```

for: biết số lần; while: điều kiện còn đúng; foreach: duyệt từng phần tử của mảng. break thoát vòng lặp; continue bỏ qua phần còn lại của lượt hiện tại. Vòng lặp lồng nhau áp dụng cho mảng nhiều chiều.

CẦN NHỚ / RỦI RO — while quên thay đổi điều kiện → lặp vô hạn. for ($i = 0; $i < count($ds); $i++) duyệt từ 0 đến n-1; không nhầm < với <=.

3.4. Mảng chỉ số, kết hợp, nhiều chiều

```
$gia = [5000, 8000];
echo $gia[0];           // 5000
echo count($gia);       // 2
$sp = ["id" => 1, "ten" => "Bút", "gia" => 5000];
echo $sp["ten"];      // Bút
$ds = [
  ["id" => 1, "ten" => "Bút"],
  ["id" => 2, "ten" => "V□"]
];
echo $ds[1]["ten"];  // V□
foreach ($ds as $item) { echo $item["ten"]; }
$thu = ["T2" => ["Sáng" => ["Toán", "Web"]]];
echo $thu["T2"]["Sáng"][1]; // Web
```

Dấu => nối khóa với giá trị trong array; [] truy cập chỉ số/khóa. PDO::FETCH_ASSOC trả hàng dạng mảng kết hợp. number_format(1250000, 0, ",", ".") → 1.250.000.

3.5. Nhúng PHP trong HTML

```
<?php $ds = [["ten" => "Bút"], ["ten" => "V□"]]; ?>
<table>
  <?php foreach ($ds as $sp): ?>
    <tr><td><?= htmlspecialchars($sp["ten"], ENT_QUOTES, "UTF-8") ?></td></tr>
  <?php endforeach; ?>
</table>
<?php if (count($ds) === 0): ?>
  <p>Không có dữ liệu</p>
<?php endif; ?>
```

<?= $x ?> tương đương <?php echo $x; ?>. Với cú pháp thay dấu ngoặc nhọn trong HTML: if(...): ... endif; và foreach(...): ... endforeach;. Dùng
 hay thẻ HTML để xuống dòng khi render, vì thuần trong source không buộc browser xuống dòng.

4. PHP NÂNG CAO — HÀM, OOP, INCLUDE, EXCEPTION (Buổi 2)

4.1. Hàm và phạm vi biến (rất hay ra đề đọc code)

```
function tinhTien($gia, $giam = 0) {
  return $gia * (100 - $giam) / 100;
}
echo tinhTien(100000);     // 100000
echo tinhTien(100000, 10); // 90000
$tong = 100;
function demo() {
  $tong = 0;  // bi□n LOCAL, không ph□i $tong phía ngoài
  $tong += 50;
}
demo(); echo $tong;        // 100
function suaTong() {
  global $tong; $tong += 50;
}
suaTong(); echo $tong;     // 150
```

Tham số mặc định dùng khi không truyền; return kết thúc hàm và trả kết quả; echo in ra output. Biến tạo trong hàm có phạm vi riêng, khác biến cùng tên bên ngoài. Slide khuyên truyền dữ liệu qua tham số/return thay vì phụ thuộc global.

```
function cong($x, $y) { return $x + $y; }
$tong = cong($tong, 50);      // cách rõ ràng, không c□n global
function dinhDangTien($n) {
  return number_format($n, 0, ",", ".") . " đ";
}
```

4.2. include, require, *_once; tách file

Cú pháp	Thiếu file / gọi lặp
include "header.php";	Cảnh báo nếu thiếu và script có thể chạy tiếp
require "db.php";	Thiếu file → lỗi nghiêm trọng, dừng
require_once "db.php";	Bắt buộc có; nếu đã nạp thì không nạp lần hai
include_once "header.php";	Như include, nhưng chỉ nạp một lần

Quy ước buổi 2/4: require_once cho kết nối/hàm dùng chung, include cho header/footer. __DIR__ lấy thư mục chứa file PHP hiện tại; hữu ích với đường dẫn tương đối đến model/common.

4.3. Class, object, visibility, $this, constructor

```
class TaiKhoan {
  public $ten;
  private $soDu = 0;
  public function __construct($ten) { $this->ten = $ten; }
  public function napTien($n) { $this->soDu += $n; }
  public function laySoDu() { return $this->soDu; }
}
$tk = new TaiKhoan("An");
$tk->napTien(500);
echo $tk->ten;         // An
echo $tk->laySoDu();  // 500
// echo $tk->soDu;    // SAI: private, không truy cập ngoài class
```

class = khuôn mẫu; object = instance tạo bằng new; property = thuộc tính; method = hàm của object; __construct tự chạy khi new; -> truy cập thuộc tính/phương thức. $this là đối tượng hiện tại. public truy cập từ ngoài được; private chỉ ở trong lớp khai báo.

CẦN NHỚ / RỦI RO — Bẫy đề mẫu: gọi $tk->soDu bên ngoài class báo lỗi truy cập private; không phải "đã nạp 500 nên in được 500". Muốn đọc an toàn: phương thức public getter, ví dụ laySoDu().

4.4. SOLID mức nhập môn & phân tầng

S: một trách nhiệm/một lý do sửa; O: mở rộng tính năng, hạn chế sửa code cũ; L: lớp con thay lớp cha được; I: không ép đối tượng triển khai việc không cần; D: phụ thuộc vào abstraction/tham số thay vì buộc cứng. Trong khóa: model quản lý dữ liệu/tính toán; view dựng HTML; kết nối PDO truyền vào hàm, tránh global.

4.5. Bắt lỗi try/catch/throw/finally

```
try {
  $gia = -2;
  if ($gia < 0) throw new Exception("Giá không hợp lệ");
  echo "Lưu thành công";
} catch (Exception $e) {
  error_log($e->getMessage()); // log kỹ thuật
  echo "Dữ liệu không hợp lệ"; // thông báo thân thiện
} finally {
  // dọn dẹp n□u c□n
}
```

CẦN NHỚ / RỦI RO — Không hiện nguyên getMessage() từ lỗi CSDL/stack/đường dẫn cho người dùng thật; ghi log chi tiết phía server. Exception có thể được throw thủ công hoặc bởi PDO khi bật ERRMODE_EXCEPTION.

5. PHP + MYSQL + CRUD (Buổi 3–4)

5.1. CSDL quan hệ và các câu SQL cốt lõi

Database gồm table; row = bản ghi; column = trường; primary key (PK) là khóa định danh duy nhất; foreign key (FK) là khóa tham chiếu bảng khác. Ví dụ: category(id) 1—n product(category_id).

```
CREATE TABLE category (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL
);
CREATE TABLE product (
  id INT AUTO_INCREMENT PRIMARY KEY,
  category_id INT NOT NULL,
  name VARCHAR(200) NOT NULL,
  price INT NOT NULL DEFAULT 0,
  FOREIGN KEY (category_id) REFERENCES category(id)
);
INSERT INTO category (name) VALUES ('Sách');
SELECT id, name, price FROM product WHERE price > 10000
ORDER BY price DESC LIMIT 5;
UPDATE product SET price = 6000 WHERE id = 1;
DELETE FROM product WHERE id = 2;
```

Câu lệnh	Cách dùng
CREATE / INSERT	Tạo bảng / chèn dữ liệu
SELECT ... WHERE	Đọc và lọc bản ghi
UPDATE ... SET ... WHERE	Thay đổi các dòng khớp WHERE
DELETE ... WHERE	Xóa dòng khớp WHERE
ORDER BY ASC / DESC	Sắp xếp tăng / giảm
LIKE "%bút%"	Tìm phần chứa chuỗi bút
LIMIT ... OFFSET ...	Phân trang
INT / VARCHAR / TEXT / DATETIME	Các kiểu: số nguyên, chuỗi có giới hạn, văn bản dài, ngày giờ
TINYINT(1) / DECIMAL(10,2)	Cờ 0/1 thường dùng và số thập phân chính xác

CẦN NHỚ / RỦI RO — UPDATE/DELETE quên WHERE có thể tác động TOÀN BỘ bảng. NOT NULL cấm NULL, UNIQUE cấm giá trị trùng, AUTO_INCREMENT tự tăng. Ví dụ dữ liệu demo mật khẩu dạng rõ ở buổi 3 phải được băm theo buổi 5.

5.2. JOIN, GROUP BY, HAVING

```
SELECT c.name, p.name
FROM category c
LEFT JOIN product p ON p.category_id = c.id;
-- giữ t□t c□ category; danh mục chưa có SP: p.name = NULL
SELECT c.name, COUNT(p.id) AS so_sp
FROM category c
LEFT JOIN product p ON p.category_id = c.id
GROUP BY c.id, c.name
HAVING COUNT(p.id) >= 5;
-- WHERE lọc b□n ghi trước khi nhóm; HAVING lọc nhóm
```

INNER JOIN: chỉ hàng có bản ghi khớp; LEFT JOIN: giữ mọi dòng phía trái; RIGHT JOIN: giữ mọi dòng phía phải, phía còn lại NULL khi thiếu. COUNT(*) đếm dòng kết quả; COUNT(cột) bỏ NULL. SUM và AVG là tổng và trung bình.

5.3. PDO, prepared statement, fetch / fetchAll

```
try {
  $pdo = new PDO(
    "mysql:host=localhost;dbname=course8;charset=utf8mb4",
    "root", ""
  );
  $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
  $pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);
  $stmt = $pdo->prepare("SELECT * FROM product WHERE id = :id");
  $stmt->execute(["id" => 5]);
  $one = $stmt->fetch();       // 1 dòng hoặc false
  $all = $pdo->query("SELECT * FROM product")->fetchAll();
} catch (PDOException $e) { error_log($e->getMessage()); }
```

PDO = PHP Data Objects; DSN chứa driver MySQL, host, database, charset. prepare tạo mẫu SQL; execute truyền riêng dữ liệu; fetch lấy một hàng; fetchAll lấy tất cả; rowCount trả số dòng bị tác động với INSERT/UPDATE/DELETE (tùy driver). PDO::FETCH_ASSOC giúp đọc dữ liệu qua khóa tên cột.

```
// CREATE - giá trị được bind, không n□i trực ti□p input vào SQL
$stmt = $pdo->prepare(
  "INSERT INTO product(name, price, category_id) VALUES(:n, :p, :c)"
);
$stmt->execute(["n" => $name, "p" => $price, "c" => $categoryId]);
// UPDATE
$pdo->prepare("UPDATE product SET price=:p WHERE id=:id")
    ->execute(["p"=>6000, "id"=>1]);
// DELETE
$pdo->prepare("DELETE FROM product WHERE id=:id")
    ->execute(["id"=>1]);
```

CẦN NHỚ / RỦI RO — Prepared statement chặn SQL Injection khi bind các GIÁ TRỊ dữ liệu. Không tự coi nội dung của input là cú pháp SQL; KHÔNG thay thế được việc escape HTML chống XSS. Placeholder không bind được tên cột/bảng tùy ý: phần đó cần danh sách cho phép.

6. PHP FORM, SESSION, COOKIE, LOGIN (Buổi 5)

6.1. Nhận form, validate phía server, htmlspecialchars

```
<?php
$loi = []; $ten = "";
if ($_SERVER["REQUEST_METHOD"] === "POST") {
  $ten = trim($_POST["ten"] ?? "");
  $gia = (int)($_POST["gia"] ?? 0);
  if ($ten === "") $loi["ten"] = "Không được đ tr ng";
  if ($gia <= 0) $loi["gia"] = "Giá ph i lớn hơn 0";
  if (count($loi) === 0) { /* lưu bằng prepared statement */ }
}
?>
<form method="post">
 <input name="ten" value="<?= htmlspecialchars($ten, ENT_QUOTES, 'UTF-8') ?>">
 <button type="submit">Lưu</button>
</form>
```

$_GET/$_POST là mảng dữ liệu request. $_SERVER["REQUEST_METHOD"] kiểm tra GET/POST. trim bỏ khoảng trắng hai đầu; (int) ép kiểu; ?? để tránh truy cập key chưa tồn tại; isset xác định key tồn tại và khác null; filter_var($email, FILTER_VALIDATE_EMAIL) kiểm tra định dạng email. Giữ giá trị nhập lại khi form lỗi, báo lỗi cạnh trường.

6.2. Session, Cookie, ID, đăng xuất

```
<?php
session_start();                 // đặt trước khi xu t nội dung
$_SESSION["user_id"] = 7;
$_SESSION["is_admin"] = 1;
$id = $_SESSION["user_id"] ?? null;
if (!isset($_SESSION["user_id"])) {
  header("Location: login.php"); exit();
}
// Sau xác thực thành công:
session_regenerate_id(true);
// Đăng xu t:
$_SESSION = [];
session_destroy();
?>
```

```
setcookie("theme", "dark", [
  "expires" => time()+86400,
  "path" => "/",
  "httponly" => true,
  "samesite" => "Lax"
]);
// Đọc cookie từ request:
$theme = $_COOKIE["theme"] ?? "light";
// Xóa cookie: đặt expires vào quá khứ, cùng path.
```

Session data giữ phía server; browser thường giữ PHPSESSID chứa Session ID. Khi request tới, browser gửi Cookie header, server tra session tương ứng. Cookie giữ phía client và có thể xem/sửa. Quyền admin phải kiểm tra bằng session ở server. setcookie thêm header phản hồi; giá trị sẽ xuất hiện ở $_COOKIE từ REQUEST SAU.

Biện pháp	Tại sao
session_start() trước output	Tránh lỗi headers already sent
session_regenerate_id(true) sau login	Giảm rủi ro session fixation: thay ID cũ
cookie HttpOnly	Chặn JavaScript đọc giá trị cookie; không tự chặn mọi XSS
cookie Secure	Chỉ gửi qua HTTPS khi cấu hình đúng
SameSite=Lax	Hạn chế một số request cross-site mang cookie
session idle/absolute timeout	Thu hẹp thời gian sử dụng session bị đánh cắp
header(Location) + exit	Redirect xong phải ngừng thực thi xử lý tiếp

6.3. Mật khẩu và xác thực đăng nhập

```
// Khi đăng ký:
$hash = password_hash($password, PASSWORD_DEFAULT);
// Lưu $hash vào cột password (VARCHAR(255) trong slide).
// Khi đăng nhập:
$st = $pdo->prepare("SELECT * FROM user WHERE username = :u");
$st->execute(["u" => trim($_POST["username"] ?? "")]);
$user = $st->fetch();
if ($user && password_verify($_POST["password"] ?? "", $user["password"])) {
  session_regenerate_id(true);
  $_SESSION["user_id"] = $user["id"];
  $_SESSION["is_admin"] = (int)$user["is_admin"];
  header("Location: index.php"); exit();
}
```

Luồng: POST credentials → validate → tìm user bằng prepared statement → password_verify với hash đã lưu → regenerate Session ID → gán thông tin cần thiết vào Session → redirect. Khi sai, thông báo chung (không tiết lộ username tồn tại hay không). Không lưu mật khẩu plaintext trong CSDL/cookie.

7. RỦI RO BẢO MẬT & LỖI THƯỜNG GẶP (Buổi 4–6)

Rủi ro	Nguyên nhân / cách phòng ngừa
SQL Injection	Ghép dữ liệu input trực tiếp vào SQL → dùng prepared statement và tham số
XSS (HTML)	In dữ liệu người dùng trực tiếp ra HTML → dùng htmlspecialchars(..., ENT_QUOTES, "UTF-8")
XSS (Vue)	v-html chèn HTML thật, nguy hiểm với dữ liệu chưa tin cậy → dùng {{ }} / v-text
Session fixation	Kẻ tấn công biết trước session ID → regenerate ID sau login
Session hijacking	Lộ ID hoặc script chạy trong phiên → escape, HttpOnly, Secure, SameSite, timeout
Privilege escalation	Tin cookie/hidden/is_admin từ client → kiểm tra quyền trên server bằng Session
Mật khẩu bị lộ	Lưu rõ hoặc trong cookie → password_hash và password_verify
Invalid form input	Chỉ kiểm tra HTML required hoặc JS → validate lại trên PHP
Mass update / delete	SQL UPDATE/DELETE thiếu WHERE → kiểm tra WHERE trước khi chạy
Information disclosure	In nguyên PDOException->getMessage() cho người dùng → log nội bộ
Vòng lặp vô hạn	while không đổi điều kiện → cập nhật biến/đặt điều kiện dừng
Cookie/Session header	echo, whitespace trước session_start/setcookie/header → headers already sent

CẦN NHỚ / RỦI RO — Ghi chú bổ sung ngoài slide: POST không có nghĩa là dữ liệu đã được mã hóa; HTTPS mới bảo vệ trên đường truyền. Các ví dụ DELETE bằng GET trong slide buổi 4 nên coi là demo luồng CRUD, không phải mẫu triển khai an toàn cho hệ thống thật (nên dùng POST + xác thực/CSRF token).

8. VUE 3: REACTIVE + DIRECTIVES (Buổi 6)

8.1. Khởi tạo Vue 3 Options API qua CDN

```
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
<div id="app">
  <input v-model="ten">
  <h2>Xin chào {{ ten }}</h2>
</div>
<script>
Vue.createApp({
  data() { return { ten: "An", gia: 5000, soLuong: 2 }; }
}).mount("#app");
</script>
```

data() trả object reactive. Khi giá trị thay đổi, Vue cập nhật phần DOM phụ thuộc tự động. Vùng #app là vùng quản lý. Interpolation {{ expression }} dùng cho nội dung text, không dùng để bind thuộc tính HTML.

8.2. Cú pháp directive quan trọng

Directive	Cách nhớ / ví dụ
{{ x }}	In text có escape; tính biểu thức {{ gia * soLuong }}
v-text="x"	Gắn nội dung dạng text, không parse HTML
v-html="x"	Parse chuỗi HTML thành DOM: không dùng input không tin cậy
v-bind:src="anh" / :src="anh"	Bind thuộc tính; :href, :class, :style, :disabled, :key
v-if / v-else-if / v-else	Thêm/xóa phần tử theo điều kiện
v-show="x"	Ẩn/hiện bằng CSS display; node vẫn ở DOM
v-for="(sp, i) in ds" :key="sp.id"	Lặp list; :key duy nhất/ổn định
v-on:click="them" / @click="them"	Bắt sự kiện và gọi method
v-model="ten"	Ràng buộc 2 chiều input↔data; các loại input/select/checkbox
v-model.number / .trim / .lazy	Chuyển số / cắt khoảng trắng / cập nhật theo change

```
<p>{{ conHang ? "Còn hàng" : "H□t hàng" }}</p>
<img :src="anh" :alt="ten">
<button :disabled="soLuong === 0">Mua</button>
<p :class="{ 'het-hang': soLuong === 0 }">{{ soLuong }}</p>
<p :style="{ color: soLuong ? 'green' : 'red' }">Trạng thái</p>
<p v-if="diem >= 8">Gi□i</p>
<p v-else-if="diem >= 5">Đạt</p>
<p v-else>Trượt</p>
<div v-show="dangTai">Đang t□i</div>
<li v-for="(sp, i) in ds" :key="sp.id">{{ i+1 }} - {{ sp.ten }}</li>
```

v-if điều kiện false thì node không còn trong DOM; v-show false chỉ đặt display:none. v-if thích hợp khi ít đổi, v-show khi bật/tắt thường xuyên. v-for nên có :key ổn định như id, không nên dùng chung v-if và v-for trên cùng một thẻ.

8.3. Event và modifier

```
<button @click="dem++">Tăng</button>
<button @click="datLai">Reset</button>
<form @submit.prevent="guiForm"><button>G□i</button></form>
<div @click="bao('cha')">
  <button @click.stop="bao('con')">B□m</button>
</div>
<input @keyup.enter="tim">
<button @click.once="chao">Ch□ chạy một l□n</button>
```

Modifier	Ý nghĩa
.prevent	event.preventDefault(), ví dụ chặn reload khi submit form
.stop	event.stopPropagation(), chặn bubble lên phần tử cha

Modifier	Ý nghĩa
.enter	Chỉ xử lý khi phím Enter
.once	Listener chỉ chạy một lần

8.4. methods, computed, watch, mounted — trọng điểm đề mẫu

```
Vue.createApp({
  data() { return { soLuong: 2, gia: 5000, canhBao: "", trangThai: "" }; },
  methods: {
    tang() { this.soLuong++; } // hành động
  },
  computed: {
    thanhTien() { return this.soLuong * this.gia; } // giá trị d□n xu□t
  },
  watch: {
    soLuong(moi, cu) { this.canhBao = moi >= 10 ? "Giá s□" : ""; }
  },
  mounted() { this.trangThai = "S□n sàng"; }
}).mount("#app");
// Trong HTML dùng: {{ thanhTien }} (KHÔNG gọi thanhTien())
```

Khối	Thời điểm / tình huống
methods	Chạy mỗi khi được gọi; hành động nút bấm, submit, xóa dòng
computed	Tính giá trị từ data; cache theo dependency, chỉ tính lại khi cần
watch	Chạy khi giá trị theo dõi đổi; side effect như cảnh báo/log/API
mounted	Hook lúc Vue đã mount xong DOM; khởi tạo, focus, nạp dữ liệu ban đầu

CẦN NHỚ / RỦI RO — Nhầm methods và computed: tính tổng tiền/lọc danh sách nên để computed; thao tác tăng/xóa nên để methods. computed chỉ trả giá trị, tránh side effect. Trong Options API dùng this.ten/this.soLuong. Data buổi 6 trong RAM trình duyệt, reload là mất.

9. CÂU HỎI NHANH TỰ ÔN THEO PHONG CÁCH ĐỀ MẪU

• Trình duyệt nhận source PHP hay HTML đã render? → HTML/JSON do server trả, không tự thực thi mã nguồn PHP.

• HTTP status 404 nằm trong request hay response? → Response.

• $x=100; function f(){ $x=0; }; f(); echo $x;? → 100: biến trong hàm là local.

• private $soDu có đọc bằng $tk->soDu từ ngoài lớp? → Không; dùng public getter.

• Thiếu MySQL XAMPP hoặc sai DSN gây gì? → PDO không kết nối thành công (thường ném PDOException).

• LEFT JOIN khác INNER JOIN? → Giữ mọi dòng ở phía trái, kể cả không khớp.

• Sau GROUP BY lọc COUNT >= 5 dùng gì? → HAVING, không phải WHERE.

• Prepared statement chặn injection như thế nào? → Giữ cấu trúc SQL độc lập với tham số dữ liệu.

• Vì sao không gửi mật khẩu trong GET? → Vì URL có thể nằm trong lịch sử, log, referrer; dùng POST qua HTTPS.

• Session và cookie lưu ở đâu? → Session server, cookie browser; browser thường lưu Session ID.

• regenerate_id sau login chặn gì? → Session fixation.

• Vue reactive là gì? → Data đổi → UI phụ thuộc tự cập nhật.

• v-model và v-html có gì khác? → v-model ràng buộc input; v-html chèn HTML, rủi ro XSS.

• computed khác methods? → Computed được cache theo dependency; methods chạy khi gọi.

10. LỘ TRÌNH ÔN TẬP KHẨN CẤP

Vòng 1: học bảng HTML/form + ký hiệu PHP ($, ., =>, ->, ===, ??, <?= ?>) + 4 khối Vue. Vòng 2: tự đoán output những đoạn code trên (không nhìn lời giải). Vòng 3: làm file Quizizz theo từng nhóm. Vòng 4: chỉ xem lại những câu sai và giải thích được vì sao đáp án khác sai.

Nguồn chính: slide Buổi 1 Tổng quan/PHP cơ bản (55 slide), Buổi 1b HTML (50 slide), Buổi 2 PHP nâng cao (49 slide), Buổi 3 MySQL (29 slide), Buổi 4 PHP–MySQL–CRUD (38 slide), Buổi 5 Form/Session/Cookie (91 slide), Buổi 6 Vue cơ bản (50 slide), và đề giữa kỳ mẫu (14 câu). Tài liệu này diễn giải lại và thêm ví dụ luyện tập; ngoại lệ/khuyến cáo mở rộng được đánh dấu “Ghi chú bổ sung ngoài slide”.